

SENAC – SERVIÇO NACIONAL DE APRENDIZAGEM COMERCIAL
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PROJETO INTEGRADOR IV
DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS
ASTROMACHINE

Aplicativo móvel para venda, montagem e personalização de computadores com temática espacial

TERCEIRA ENTREGA – FASE 3

Funcionalidade Principal e Gerenciamento de Riscos

Integrantes:

Guilherme da Silva Serafim
Gustavo Magalhães Prada de Castro

São Paulo – 2026

SUMÁRIO

1. INTRODUÇÃO	3
2. OBJETIVOS DO PROJETO	4
3. ESCOPO E CONTEXTO DO ASTROMACHINE	5
4. HISTÓRICO – FASE 2 (REVISÃO).....	6
5. ARQUITETURA ATUAL DO SISTEMA.....	7
6. BANCO DE DADOS – SQLite.....	8
7. IMPLEMENTAÇÃO DO CRUD (Item 3.1)	10
8. INTEGRAÇÃO COM API E SINCRONIZAÇÃO (Item 3.2)	11
9. GERAÇÃO DE DADOS EM JSON PARA DASHBOARD (Item 3.3)	12
10. PLANO DE COMUNICAÇÃO E RELATÓRIO DE STATUS (Item 3.4)	13
11. GERENCIAMENTO DE RISCOS ATUALIZADO	15
12. EVIDÊNCIAS – PRINTS DAS TELAS	16
13. CONCLUSÃO	34
14. BIBLIOGRAFIA	35
15. ANEXOS.....	36

1. INTRODUÇÃO

O presente documento corresponde à Terceira Entrega (Fase 3) do Projeto Integrador IV – Desenvolvimento para Dispositivos Móveis, do curso Tecnologia em Análise e Desenvolvimento de Sistemas do Senac. Nesta etapa, o projeto AstroMachine consolida sua funcionalidade principal por meio da implementação de operações CRUD (Criar, Ler, Atualizar e Deletar), da integração entre o aplicativo móvel e o servidor backend através de uma API RESTful, da persistência de dados em banco SQLite e da geração de dados em formato JSON para apoio à disciplina de Análise de Dados e Estatística.

Em continuidade ao trabalho desenvolvido na Fase 2, em que foram entregues a configuração do ambiente, as primeiras telas, o Diagrama de Entidade-Relacionamento (DER), a configuração inicial do backend em Node.js e o plano de mitigação de riscos, esta entrega apresenta o sistema funcional, com fluxos completos de cadastro, login, consulta de catálogo, gestão de carrinho, checkout simulado, gerenciamento de produtos via painel administrativo e exportação de dados consolidados.

O documento preserva o histórico das atividades realizadas na Fase 2, agora revisado à luz das decisões técnicas adotadas, e detalha as novas seções correspondentes aos itens 3.1 a 3.4 do roteiro oficial. Inclui ainda a atualização do gerenciamento de riscos, evidências de execução, relatório de status do backlog e a Sprint Review da Fase 3.

2. OBJETIVOS DO PROJETO

2.1 Objetivo geral

Desenvolver um aplicativo móvel funcional, denominado AstroMachine, voltado à venda, montagem e personalização de computadores com temática espacial, integrando interface mobile, backend RESTful e banco de dados persistente, em conformidade com os requisitos pedagógicos do Projeto Integrador IV.

2.2 Objetivos específicos da Fase 3

- Implementar operações CRUD completas (criar, ler, atualizar e deletar) para a entidade Produto, contemplando o painel administrativo no aplicativo e as rotas correspondentes no backend.
- Disponibilizar a API RESTful em Node.js + Express, permitindo a sincronização de dados entre o aplicativo móvel e o servidor.
- Substituir o armazenamento volátil em memória por um banco de dados SQLite persistente, garantindo a continuidade dos dados entre execuções.
- Garantir o consumo da API a partir do app mobile nos fluxos de login, cadastro, catálogo, carrinho e checkout simulado.
- Gerar arquivo JSON consolidado contendo indicadores e registros para apoio à disciplina de Análise de Dados e Estatística.
- Conduzir Sprint Review da Fase 3, gerar relatório de status do projeto e atualizar a tabela de riscos com as mitigações efetivamente aplicadas.
- Manter os critérios de acessibilidade já contemplados na fase anterior, oferecendo opções de fonte ampliada, alto contraste e logout simples.

3. ESCOPO E CONTEXTO DO ASTROMACHINE

O AstroMachine é um aplicativo móvel desenvolvido em React Native com Expo, destinado a uma loja virtual fictícia especializada em computadores e periféricos com identidade visual de temática espacial. A proposta integra três pilares: (i) venda de produtos prontos, (ii) personalização e montagem de máquinas conforme o perfil do cliente e (iii) agendamento de serviços de manutenção e upgrade.

O sistema é composto por dois grandes módulos que se comunicam por uma API RESTful: o aplicativo móvel (cliente), responsável pela interface e pela experiência do usuário, e o servidor backend, responsável pela persistência de dados, regras de negócio e exposição dos endpoints. Esta arquitetura cliente-servidor foi mantida desde a Fase 2 e amadurecida na Fase 3 com a adoção de SQLite como mecanismo de persistência.

Está dentro do escopo da Fase 3

- Cadastro e autenticação de usuários (cliente e administrador).
- Catálogo de produtos consumido a partir do backend.
- Carrinho de compras com persistência em sessão do aplicativo.
- Checkout simulado, com registro do pedido no banco de dados.
- Painel administrativo com CRUD de produtos.
- Tela de acessibilidade e logout.
- Geração de arquivo JSON com dados consolidados.

Está fora do escopo da Fase 3

- Integração com gateway real de pagamento (ex.: Mercado Pago, PagSeguro). O checkout permanece simulado, suficiente para validar o fluxo de compra e gerar registros de pedido.
- Publicação em lojas oficiais (Google Play / App Store), prevista para fases posteriores.
- Notificações push e relatórios analíticos avançados, que serão tratados na Fase 4.

4. HISTÓRICO – FASE 2 (REVISÃO)

Esta seção preserva o conteúdo essencial entregue na Fase 2 e o atualiza onde decisões da Fase 3 modificaram o estado original. A Fase 2 concentrou-se na configuração do ambiente de desenvolvimento, na criação da arquitetura inicial de código, na estrutura de dados e na primeira rodada de identificação e mitigação de riscos.

4.1 Configuração do Ambiente e Interface Inicial

Foi configurado o ambiente de desenvolvimento utilizando o Visual Studio Code como IDE principal, Node.js como runtime, npm para o gerenciamento de pacotes e Expo para a execução e teste do aplicativo em dispositivos físicos e emuladores. O projeto base do aplicativo foi criado em React Native com Expo, organizado em pastas para telas, componentes, serviços e contexto de navegação.

As telas iniciais foram implementadas seguindo a identidade visual de temática espacial do AstroMachine, com paleta de cores escura, elementos gráficos referenciando galáxias e tipografia tecnológica. Foram entregues, ao final da Fase 2, as telas de Login, Cadastro, Catálogo, Carrinho, Checkout e Acessibilidade, todas compilando e iniciando corretamente no Expo Go e em emulador Android.

4.2 Modelagem e Estrutura do Banco de Dados

Na Fase 2 foi elaborado o Diagrama de Entidade-Relacionamento (DER), contemplando as entidades principais do domínio: Usuários, Produtos, Serviços e Agendamentos. A configuração inicial do backend foi realizada em Node.js com o framework Express, expondo um conjunto preliminar de rotas para autenticação e listagem de produtos, ainda com armazenamento em memória.

A entrega original previa a possibilidade de uso de SQLite como banco mobile local. Esta decisão foi revisada e amadurecida na Fase 3: o SQLite passou a residir no backend, atuando como fonte central de dados, enquanto o aplicativo móvel passou a consumi-los via API. Esta evolução está descrita em detalhe nas seções 5 e 6 deste documento.

4.3 Monitoramento e Sprint Review – Fase 2

Foi realizada a primeira Reunião de Revisão da Sprint, com os integrantes do grupo apresentando as interfaces iniciais e discutindo lições aprendidas. Os principais ajustes acordados foram a padronização visual entre telas, a melhoria de mensagens de erro nos formulários e a antecipação da decisão de tornar o backend o ponto único de verdade para os dados, decisão posteriormente concretizada com o SQLite na Fase 3.

4.4 Análise e Mitigação de Riscos – Fase 2

Na Fase 2 foram identificados e classificados os principais riscos do projeto, com destaque para indisponibilidade da API durante demonstrações, perda de dados em armazenamento volátil, dificuldades de conexão entre o Expo Go e o backend em redes locais e a impossibilidade de uso de gateway real de pagamento dentro do prazo do semestre. Para cada risco foi definido um Plano B inicial, posteriormente refinado e efetivado na Fase 3, conforme descrito na seção 11.

5. ARQUITETURA ATUAL DO SISTEMA

A arquitetura do AstroMachine na Fase 3 segue o modelo cliente-servidor, organizado em três camadas principais: (i) aplicativo móvel em React Native/Expo, (ii) backend em Node.js + Express e (iii) banco de dados SQLite persistente. O backend funciona como fonte central de dados (single source of truth), e o aplicativo móvel consome essas informações por meio de uma API RESTful.

Componentes e responsabilidades

- Aplicativo móvel (React Native + Expo): responsável pela apresentação, navegação, validação de formulários, gestão de estado em sessão e consumo das rotas do backend. Reúne as telas de Login, Cadastro, Catálogo, Carrinho, Checkout, Acessibilidade e Painel Administrativo.
- Backend (Node.js + Express): expõe endpoints REST para autenticação, manipulação de produtos, registro de pedidos, listagem de serviços e agendamentos. Aplica as regras de negócio e abstrai o acesso ao banco.
- Banco de dados (SQLite): armazena de forma persistente as entidades do sistema. O arquivo principal está localizado em `server/data/astromachine.db`.

Estratégia de descoberta de URL da API

Para reduzir o atrito de configuração entre os ambientes possíveis (dispositivo físico via Expo Go, emulador Android e execução em localhost), o aplicativo realiza tentativa de detecção automática de URL da API a partir de uma lista de candidatos pré-configurados. Caso a primeira URL não responda dentro de um intervalo de tempo, o aplicativo passa para a próxima alternativa, garantindo maior probabilidade de sucesso na conexão durante demonstrações em sala.

Plano de contingência: fallback local

Para mitigar o risco de indisponibilidade da API durante demonstrações, o aplicativo móvel mantém um fallback local com dados de exemplo de produtos e serviços. Caso todas as URLs candidatas falhem, o app continua exibindo o catálogo e permitindo a navegação pelas principais telas em modo demonstrativo. Este fallback não substitui a operação plena com backend, mas assegura a continuidade da experiência durante a apresentação.

Tecnologias e bibliotecas principais

- React Native com Expo SDK – framework e plataforma de execução do app móvel.
- React Navigation – gestão de rotas e fluxo entre telas.
- Node.js + Express – servidor HTTP e roteamento da API.
- better-sqlite3 / sqlite3 – driver para acesso ao banco SQLite.
- VS Code – IDE de desenvolvimento.
- Git – controle de versão.

6. BANCO DE DADOS – SQLite

O AstroMachine utiliza SQLite como banco de dados relacional persistente, residente no servidor backend. A escolha foi pautada pela simplicidade de configuração, ausência de processo de servidor separado, portabilidade e adequação ao porte de uma aplicação acadêmica. O arquivo principal do banco fica armazenado em `server/data/astromachine.db` e é versionado em conjunto com a estrutura de migração e o seed inicial.

A persistência em SQLite substituiu o armazenamento volátil utilizado no início da Fase 2, eliminando o risco de perda de dados a cada reinício do servidor e permitindo a manutenção de estado entre execuções, requisito essencial para validação dos itens 3.1 e 3.2 do roteiro.

Tabelas implementadas

Tabela	Finalidade
users	Cadastro de usuários (clientes e administradores), com credenciais de acesso e perfil.
products	Catálogo de produtos disponíveis no AstroMachine (computadores, periféricos e itens com temática espacial).
services	Serviços de personalização e manutenção oferecidos (montagem, upgrade, limpeza, etc.).
appointments	Agendamentos realizados por clientes, vinculando usuário, possivelmente um produto e uma data.
appointment_services	Tabela associativa que relaciona agendamentos aos serviços contratados (relação N:N).
orders	Pedidos gerados durante o checkout simulado, contendo o usuário comprador, data e total.
order_items	Itens de cada pedido, referenciando produtos e quantidades.

Relacionamentos principais

- Um usuário (users) pode ter vários pedidos (orders) e vários agendamentos (appointments).
- Um pedido (orders) possui vários itens (order_items).
- Cada item de pedido (order_items) referencia um produto (products) e armazena a quantidade adquirida.
- Um agendamento (appointments) pode envolver um produto (products) e um ou mais serviços (services), por meio da tabela associativa appointment_services.

Dados iniciais (seed)

Para apoiar testes e demonstrações, o banco é populado, no primeiro arranque, com um conjunto de dados controlado:

- Usuário administrador: admin@astromachine.com / admin123.
- Cliente de exemplo: guilherme@email.com / cliente123.
- 6 produtos com identidade visual de temática espacial.
- 5 serviços de personalização.
- 2 agendamentos de exemplo.

Esses dados permitem que o aplicativo seja apresentado em condições reprodutíveis. A documentação técnica complementar do esquema está disponível no arquivo docs/fase3-database-schema.md, presente no repositório do projeto.

7. IMPLEMENTAÇÃO DO CRUD (Item 3.1)

Em conformidade com o item 3.1 do roteiro da Fase 3, o AstroMachine implementa as quatro operações CRUD – Criar, Ler, Atualizar e Deletar – sobre os dados persistidos no banco SQLite. A entidade escolhida como CRUD principal foi Produto (products), por ser o eixo central do e-commerce e por permitir a demonstração completa do ciclo de vida de um registro, desde o cadastro até a exclusão.

Painel administrativo no aplicativo

O aplicativo móvel disponibiliza um painel administrativo, acessível mediante autenticação com credenciais de administrador, no qual é possível:

- Listar todos os produtos cadastrados no backend (Read).
- Cadastrar um novo produto, informando nome, descrição, preço, imagem e categoria (Create).
- Editar os campos de um produto existente, com persistência das alterações no banco (Update).
- Remover produtos do catálogo, com confirmação prévia (Delete).

Cada ação realizada no painel resulta em uma chamada HTTP correspondente à API REST do backend, que valida os dados, aplica as regras de negócio e atualiza o banco SQLite. Após a resposta de sucesso, a tela do painel é recarregada para refletir o novo estado dos dados.

Rotas REST utilizadas no CRUD de produtos

Verbo / Rota	Operação	Descrição
GET /products	Read	Lista todos os produtos cadastrados no backend.
GET /products/:id	Read	Recupera os detalhes de um produto específico.
POST /products	Create	Cadastra um novo produto no banco SQLite.
PUT /products/:id	Update	Atualiza os dados de um produto existente.
DELETE /products/:id	Delete	Remove um produto do banco SQLite.

Validações e tratamento de erros

As operações de criação e atualização aplicam validações simples, como obrigatoriedade dos campos principais e formato numérico do preço. O backend retorna códigos HTTP apropriados (200, 201, 400, 404, 500), e o aplicativo trata as respostas exibindo mensagens claras ao administrador, mantendo a coerência entre os estados local e remoto.

8. INTEGRAÇÃO COM API E SINCRONIZAÇÃO (Item 3.2)

O item 3.2 do roteiro determina a implementação do consumo de serviços RESTful para garantir a sincronização correta entre o app móvel e o backend. No AstroMachine, esta integração foi concretizada por meio de uma camada de serviços no aplicativo (services), responsável por encapsular as chamadas HTTP e tratar as respostas de forma centralizada.

Fluxos integrados ao backend

- Login: o aplicativo envia as credenciais para o endpoint de autenticação. Em caso de sucesso, recebe um identificador de sessão simples e os dados do usuário, que ficam disponíveis no contexto global do app.
- Cadastro: novos usuários são criados via API, com persistência imediata na tabela users do SQLite.
- Catálogo: a lista de produtos exibida ao cliente é obtida diretamente do backend, refletindo o estado atual da tabela products.
- Carrinho: o carrinho é mantido em estado de sessão no aplicativo. No momento do checkout, seus itens são enviados ao backend para criação do pedido.
- Checkout simulado: ao confirmar o pedido, o backend insere um registro em orders e os respectivos itens em order_items. O pagamento real está fora do escopo do projeto; o checkout simulado é suficiente para validar o fluxo mobile e gerar registros consistentes para análise estatística.

Padrão das chamadas RESTful

Todas as chamadas seguem o padrão REST: rotas baseadas em recursos, verbos HTTP semânticos (GET, POST, PUT, DELETE) e troca de dados em JSON. O aplicativo utiliza fetch ou axios para realizar as requisições, com tratamento de timeout e captura de erros de rede. Cada serviço expõe funções específicas (por exemplo, getProducts, createProduct, login, register, createOrder), o que mantém o restante do código desacoplado dos detalhes de comunicação.

Sincronização entre app e backend

A sincronização é orientada por eventos de tela: ao abrir o catálogo, o app busca a lista atualizada de produtos; ao concluir o checkout, atualiza o backend imediatamente. Não há cache persistente local de dados de domínio, o que garante que o backend permaneça como fonte única da verdade. Esta decisão simplifica o modelo e evita conflitos de versão no escopo desta entrega.

Plano de contingência

Como já apresentado na seção 5, o aplicativo possui um fallback local com dados de demonstração. Quando a API está indisponível, o app preserva a navegação principal exibindo o catálogo de exemplo e indicando ao usuário que está em modo demonstrativo. Esta funcionalidade é essencial para reduzir o impacto de falhas de rede ou indisponibilidade do servidor durante apresentações.

9. GERAÇÃO DE DADOS EM JSON PARA DASHBOARD (Item 3.3)

Para atender ao item 3.3 do roteiro, em conjunto com a disciplina de Análise de Dados e Estatística, o projeto entrega um arquivo consolidado em formato JSON, contendo indicadores e registros que servirão de insumo para a elaboração de um dashboard analítico.

O arquivo é gerado pelo backend a partir das tabelas do SQLite e está disponível em docs/fase3-dashboard-data.json, no repositório do projeto. Ele é construído de forma a fornecer tanto registros detalhados quanto agregados úteis para visualização.

Conteúdo do arquivo JSON

- Lista de produtos cadastrados, com identificador, nome, categoria e preço.
- Lista de serviços de personalização disponíveis.
- Lista de usuários (sem credenciais), com perfil e data de cadastro.
- Pedidos registrados pelo checkout simulado, incluindo total, data e quantidade de itens.
- Agendamentos cadastrados, com produto associado e serviços vinculados.
- Indicadores agregados, como total de produtos por categoria, total de pedidos por período, ticket médio simulado e número de agendamentos por serviço.

Aplicação na disciplina de Análise de Dados

O JSON foi estruturado para ser facilmente carregado em ferramentas de análise (planilhas, Power BI ou notebooks Python). A combinação de registros detalhados e agregados permite a construção de gráficos comparativos, séries temporais e análises de distribuição, em alinhamento com os requisitos da disciplina parceira.

10. PLANO DE COMUNICAÇÃO E RELATÓRIO DE STATUS (Item 3.4)

10.1 Plano de comunicação

Durante a Fase 3, a equipe manteve reuniões periódicas presenciais e remotas, complementadas pelo uso de aplicativo de mensagens para comunicação assíncrona e por um repositório Git compartilhado para sincronização do código-fonte. As decisões técnicas relevantes foram registradas em arquivos de documentação dentro da pasta docs do projeto, garantindo rastreabilidade ao longo da entrega.

- Reuniões semanais entre os integrantes para alinhamento e priorização do backlog.
- Comunicação assíncrona por mensagens para dúvidas pontuais e validações rápidas.
- Repositório Git para versionamento, revisão de código e histórico de alterações.
- Documentação complementar em Markdown na pasta docs/.

10.2 Sprint Review – Fase 3

Foi realizada Sprint Review ao final do ciclo da Fase 3, com a apresentação dos seguintes incrementos: persistência em SQLite, CRUD completo de produtos no painel administrativo, integração efetiva entre app e backend para login, cadastro, catálogo, carrinho e checkout simulado, geração do arquivo JSON para dashboard e validação do fallback local. As lições aprendidas foram documentadas em docs/fase3-status-report.md.

10.3 Relatório de status do projeto

O backlog obrigatório da Fase 3 foi concluído integralmente, conforme apresentado na tabela a seguir.

Item do roteiro	Descrição	Status
3.1 CRUD	Criar, ler, atualizar e deletar produtos via painel admin e API.	Concluído
3.2 Integração / API	Login, cadastro, catálogo, carrinho e checkout consumindo a API REST.	Concluído
3.3 JSON dashboard	Geração de docs/fase3-dashboard-data.json para Análise de Dados.	Concluído
3.4 Comunicação / Status	Sprint Review da Fase 3 e relatório de status.	Concluído
Persistência (SQLite)	Migração de dados em memória para SQLite em server/data/astromachine.db.	Concluído
Acessibilidade	Tela de fonte ampliada, alto contraste e logout.	Concluído

Item do roteiro	Descrição	Status
Fallback local	Dados de demonstração no app caso a API esteja indisponível.	Concluído

Percentual de conclusão

Considerando os itens obrigatórios definidos no roteiro oficial da Fase 3 (3.1, 3.2, 3.3 e 3.4), o percentual de conclusão é de 100%. Os itens auxiliares (persistência em SQLite, acessibilidade e fallback local) também foram concluídos, reforçando a robustez da entrega.

Impedimentos observados

- Dificuldade ocasional de conexão entre o Expo Go e o backend, especialmente em redes Wi-Fi com isolamento de clientes. Mitigação aplicada: detecção automática de URL e fallback local.
- Tempo limitado para a integração com gateway real de pagamento, decisão tomada na Fase 2 de manter o checkout simulado dentro do escopo.
- Curva de aprendizado em consultas SQL específicas, contornada com testes incrementais e revisão entre os integrantes.

11. GERENCIAMENTO DE RISCOS ATUALIZADO

A tabela a seguir consolida os riscos identificados desde a Fase 2 e atualiza o estado das mitigações efetivamente aplicadas na Fase 3. Cada risco é classificado por nível de impacto e acompanhado da contramedida adotada pela equipe.

Risco	Impacto	Mitigação aplicada
API indisponível durante a demonstração	Alto	Implementação de fallback local no app móvel, com dados de demonstração para garantir continuidade da apresentação.
Perda de dados em armazenamento volátil	Alto	Substituição do armazenamento em memória por SQLite persistente em server/data/astromachine.db.
Dificuldade de conexão Expo Go ↔ backend	Médio	Detecção automática de URL da API a partir de uma lista de candidatos (localhost, emulador, IP da rede).
Pagamento real fora do escopo	Médio	Adoção de checkout simulado, suficiente para validar o fluxo mobile e gerar registros de pedido para análise.
Necessidade de dados para dashboard	Médio	Geração do arquivo docs/fase3-dashboard-data.json, com registros e agregados consumíveis pela disciplina de Análise de Dados.
Conflito de versões no repositório	Baixo	Uso disciplinado de branches e revisão entre os integrantes antes de integrar na branch principal.
Curva de aprendizado em SQL	Baixo	Testes incrementais e documentação do esquema em docs/fase3-database-schema.md.

As mitigações descritas foram testadas em cenários simulados durante o desenvolvimento, em particular o fallback local e a detecção automática de URL, ambos validados durante a Sprint Review da Fase 3.

12. EVIDÊNCIAS – PRINTS DAS TELAS

Esta seção é destinada à inserção dos prints das telas e dos artefatos do projeto. As imagens devem ser inseridas na ordem indicada na tabela abaixo, preferencialmente com legenda numerada (Figura 1, Figura 2, ...) e breve descrição.

Tela / Evidência	O que demonstra
Tela de Login	Autenticação via API; entrada de credenciais e retorno de sessão.
Tela de Cadastro	Criação de usuário no backend, persistido em SQLite.
Catálogo	Listagem dinâmica de produtos consumida da API.
Carrinho	Itens adicionados pelo usuário, com cálculo de total.
Checkout simulado	Confirmação do pedido e registro em orders / order_items.
Acessibilidade	Fonte ampliada, alto contraste e ajustes visuais.
Sair da conta	Encerramento de sessão e retorno à tela de login.
Painel administrativo (CRUD)	Listagem, criação, edição e exclusão de produtos.
Arquivo JSON / Banco SQLite	Estrutura dos dados gerados em docs/fase3-dashboard-data.json e tabelas em astromachine.db.

Figura 1 – Tela de Login



AstroMachine

Seu PC dos sonhos, feito sob medida

Email

 seu@email.com

Senha



.....



[Esqueci minha senha](#)

Entrar

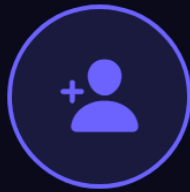
Não tem conta? [Cadastre-se](#)

Demo

Admin: admin@astromachine.com / admin123

Cliente: guilherme@email.com / cliente123

Figura 2 – Tela de Cadastro



Criar Conta

Junte-se ao universo AstroMachine

Nome completo

 Seu nome

Email

 seu@email.com

Senha



.....



Confirmar senha



.....

Cadastrar

Já tem conta? [Entrar](#)

Figura 3 – Catálogo

Olá, Guilherme 🚀

Catálogo



🌐 Modo offline — dados locais



Build Andromeda

PC gamer de alto desempenho com temática da galáxia ...

R\$ 8.999,90



PC Orion

Workstation robusta inspirada na constelação de Orion. Ide...

R\$ 12.499,90



Nebula Prime

Build intermediária com visual nebuloso e LEDs RGB ...

R\$ 5.999,90



Exoplaneta X

Setup compacto Mini-ITX com design futurista inspirado em...

R\$ 7.499,90



Supernova Elite

O top de linha da



Início



Carrinho

Figura 4 – Carrinho



Carrinho



Build Andromeda

R\$ 8.999,90



−

1

+

Subtotal

R\$ 8.999,90

Finalizar Compra →



Início



Carrinho

Figura 5 – Checkout



Checkout

Resumo do Pedido

✓ Build Andromeda x1 R\$ 8.999,90

Total **R\$ 8.999,90**

Método de Pagamento



Crédito



Débito



PIX

Dados do Cartão (Simulação)

Nome no cartão

Nome completo

Número do cartão

0000 0000 0000 0000

Validade

MM/AA

CVV

123

Figura 6 – Acessibilidade



Acessibilidade

Configure as opções de acessibilidade para melhorar sua experiência no AstroMachine.

Aa

Fonte Grande

Aumenta o tamanho de todos os textos do app para melhor legibilidade.



Alto Contraste

Aumenta o contraste das cores para facilitar a visualização dos elementos.



O AstroMachine foi desenvolvido seguindo diretrizes de acessibilidade, incluindo suporte a leitores de tela, contraste adequado e áreas de toque confortáveis (mínimo 44px).



Sair da conta

Figura 7 – Sair da conta



Acessibilidade

Configure as opções de acessibilidade para melhorar sua experiência no AstroMachine.

Aa

Fonte Grande

Aumenta o tamanho de todos os textos do app para melhor legibilidade.



Alto Contraste

Aumenta o contraste das cores para facilitar a visualização dos elementos.



O AstroMachine foi desenvolvido seguindo diretrizes de acessibilidade, incluindo suporte a leitores de tela, contraste adequado e áreas de toque confortáveis (mínimo 44px).



Sair da conta

Figura 8 – Painei administrativo / CRUD



Painel Admin



6

Produtos

6

Em Estoque



Build Andromeda

R\$ 8.999,90

high-end



PC Orion

R\$ 12.499,90

workstation



Nebula Prime

R\$ 5.999,90

mid-range



Exoplaneta X

R\$ 7.499,90

compact



Supernova Elite

R\$ 19.999,90

extreme



Cosmos Starter

R\$ 3.499,90

entry



Figura 9 – Trecho do JSON / SQLite

docs/fase3-dashboard-data.json

Trecho do JSON exportado a partir do banco SQLite do AstroMachine

```
{
  "generatedAt": "2026-04-28T14:24:16.129Z",
  "summary": {
    "totalProducts": 6,
    "totalServices": 5,
    "totalUsers": 3,
    "totalOrders": 0,
    "revenue": 0,
    "productsInStock": 6,
    "productsOutOfStock": 0
  },
  "products": [
    {
      "id": "1bcf3a0c-7e35-4924-9677-232ac02e6ccf",
      "name": "Build Andromeda",
      "description": "PC gamer de alto desempenho com tematica da galaxia Andromeda. RTX 4070, Ryzen 7 7800X3D, 32GB DDR5.",
      "price": 8999.9,
      "imageUrl": "",
      "category": "high-end",
      "specs": "RTX 4070 | Ryzen 7 7800X3D | 32GB DDR5 | 1TB NVMe | Water Cooler 240mm",
      "inStock": true,
      "createdAt": "2026-04-27T23:02:57.857Z"
    },
    {
      "id": "470e2dc4-918c-4eec-bc0a-8a500b6594d4",
      "name": "PC Orion",
      "description": "Workstation robusta inspirada na constelacao de Orion. Ideal para criadores de conteudo e
```

13. CONCLUSÃO

A Fase 3 consolidou a funcionalidade principal do AstroMachine, integrando o aplicativo móvel em React Native/Expo, o backend em Node.js + Express e um banco de dados SQLite persistente em uma única solução coerente. As operações CRUD foram entregues sobre a entidade central do domínio (produtos), validando os fluxos completos de criação, leitura, atualização e exclusão por meio do painel administrativo e das rotas REST correspondentes.

A integração entre app e backend permitiu que login, cadastro, catálogo, carrinho e checkout simulado passassem a operar contra a API, com sincronização orientada a eventos de tela. A geração do arquivo JSON para a disciplina de Análise de Dados e Estatística completa o ciclo da entrega, fornecendo insumo direto para a construção de dashboards analíticos.

As decisões técnicas adotadas – em especial a persistência em SQLite, o fallback local no app e a detecção automática de URL – mostraram-se eficazes na mitigação dos riscos previstos desde a Fase 2. A equipe encerra esta fase com 100% dos itens obrigatórios concluídos e em condições adequadas para iniciar a Fase 4, dedicada ao polimento, à documentação final, ao vídeo de apresentação e à entrega definitiva do Projeto Integrador IV.

14. BIBLIOGRAFIA

EXPO. Expo Documentation. Disponível em: <https://docs.expo.dev>. Acesso em: abr. 2026.

REACT NATIVE. React Native Documentation. Disponível em: <https://reactnative.dev/docs/getting-started>. Acesso em: abr. 2026.

NODE.JS FOUNDATION. Node.js Documentation. Disponível em: <https://nodejs.org/en/docs>. Acesso em: abr. 2026.

EXPRESS. Express – Node.js web application framework. Disponível em: <https://expressjs.com>. Acesso em: abr. 2026.

SQLITE. SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: abr. 2026.

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures. 2000. Tese de Doutorado – University of California, Irvine.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

SCHWABER, K.; SUTHERLAND, J. The Scrum Guide. Scrum.org, 2020.

15. ANEXOS

Anexo A – Estrutura de pastas do projeto

- server/data/astromachine.db – arquivo principal do banco SQLite.
- server/ – código-fonte do backend em Node.js + Express.
- app/ ou src/ – código-fonte do aplicativo React Native/Expo.
- docs/fase3-dashboard-data.json – arquivo JSON gerado para a disciplina de Análise de Dados.
- docs/fase3-database-schema.md – documentação textual do esquema do banco.
- docs/fase3-status-report.md – relatório de status da Fase 3.

Anexo B – Credenciais de demonstração

- Administrador: admin@astromachine.com / admin123.
- Cliente: guilherme@email.com / cliente123.

Anexo C – Endpoints REST principais

- GET /products – lista todos os produtos.
- GET /products/:id – retorna um produto específico.
- POST /products – cria novo produto (admin).
- PUT /products/:id – atualiza produto existente (admin).
- DELETE /products/:id – remove produto (admin).
- POST /login – autenticação de usuário.
- POST /register – cadastro de novo usuário.
- POST /orders – registra pedido do checkout simulado.